



PING PONG GAME



احمد حمدي الاصور

احمد ضياء احمد يسري

ايار محمد هشام

محمد مظهر السيد حبشي

وعد جمال امام

يارا عادل عبدالفتاح

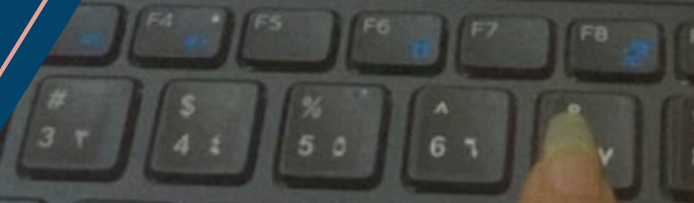
يوسف محمد أبو الخير

```
# start position
# Things don't draw lines while moving

dx, ball_dy = 1, 1
ball_speed = .2

#center line
center_line = turtle.Turtle()
center_line.speed(0)
center_line.shape("square")
center_line.color("white")
center_line.shapesize(stretch_wid=25, stretch_len=1)
center_line.penup()
center_line.goto(x=0, y=0)

# player 1
```





PING PONG GAME



BY

AHMED DIAA YOSRY
AHMED HAMDY ALASWAR
EYAD MOHAMED HESHAM
MOHAMED MAZHAR HEBASHEY
WAAD GAMAL EMAM
YARA ADEL ABDELFAHATTAH
YOUSSEF MOHAMED ABO ELKHIER

PRESENTED TO

PROF.Dr.NADA EL-SHNNAWY
PROF.Dr.MOHAMED SHOAIB

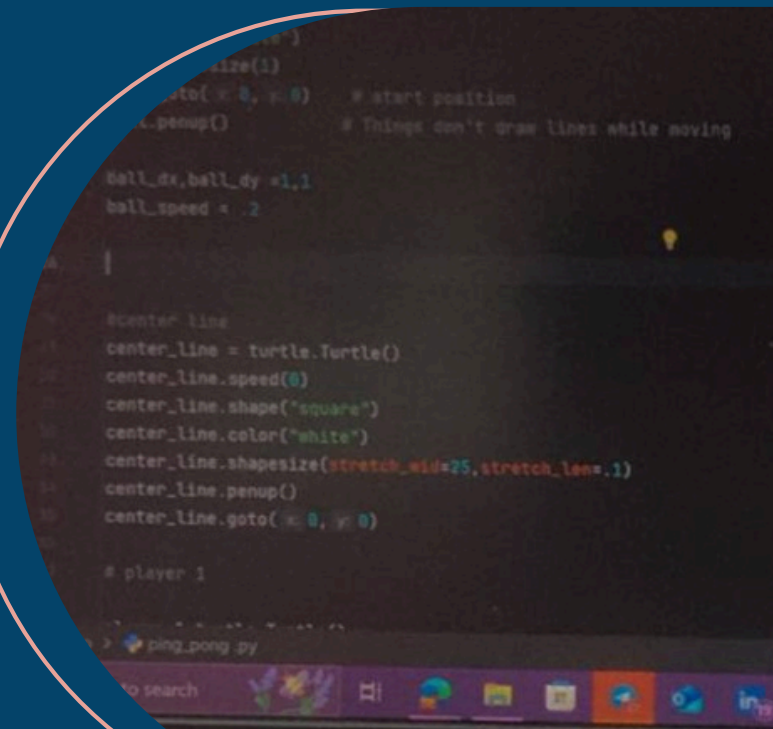




Table of Contents

01

Introduction

02

Objective

03

System overview

04

Tools and
technologies

05

Code explanation

06

Challenges and
solutions

07

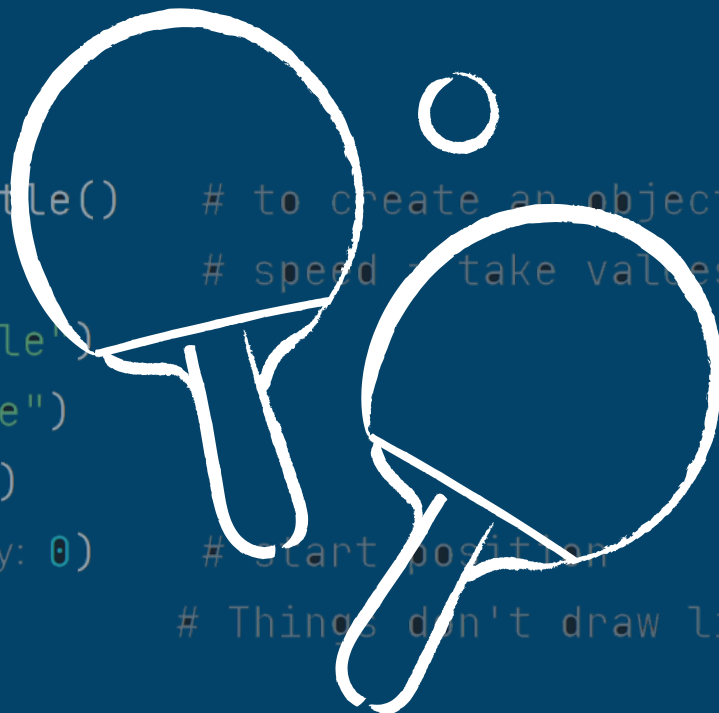
Conclusion

INTRODUCTION

This project aims to design and implement a simple two-player Ping Pong game using the Python programming language along with the Turtle graphics library.

The primary goal is to create an interactive and enjoyable game while developing a deeper understanding of graphical programming, event handling, and basic game mechanics in Python.

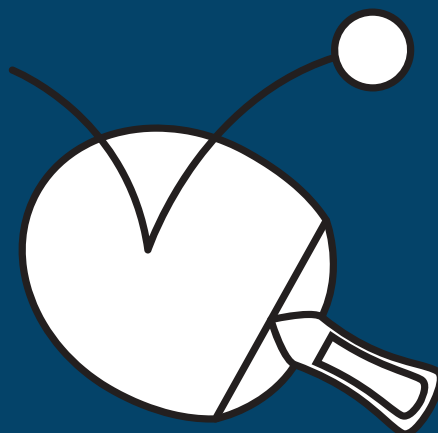
This project is particularly useful for beginners who want to learn the basics of game development using Python in a fun and practical way.



OBJECTIVE

The primary objective of this project is to design and implement an interactive two-player Ping Pong game using Python. Each player controls a paddle to rally the ball across the screen, aiming to prevent it from passing their side. The game simulates real-time motion, collision detection, and score tracking to create a smooth and engaging gameplay experience.

PLAYER	WINS	LOSSES	HIGHEST SCORE
Player1	15	10	120
Player2	10	15	110



SYSTEM OVERVIEW

It includes the following main components:

- **Two paddles:** one on the left and one on the right side of the screen—each controlled by a player

- **A ball:** that moves across the screen, bouncing off the top and bottom walls, and reversing direction when it hits a paddle.

- **A scoring system:** that tracks how many points each player scores. A player earns a point when the opponent misses the ball.

- **Keyboard controls:** that allow each player to move their paddle up and down smoothly.

```
objects ===  
    ball = turtle.Turtle()  
    ball.speed(0) # drawing speed  
    ball.shape("square")  
    ball.color("white")  
    # scale factor * default size (20)  
    ball.shapesize(stretch_len=1, stretch_wid=1)  
    ball.goto(x=0, y=0) # start position  
    ball.penup()  
  
    # game loop =====  
    while True:  
        window.update()
```

TOOLS AND TECHNOLOGIES

- Python 3.10
 - Turtle Library
 - PY CHARM (IDE)
 - Windows 11
- Operating System



CODE EXAPLANATION

01

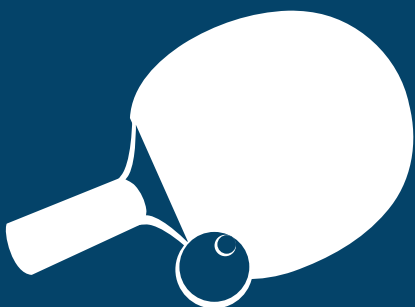
WINDOW

02

PLAYER 1&2

03

BALL



04

CENTER LINE

05

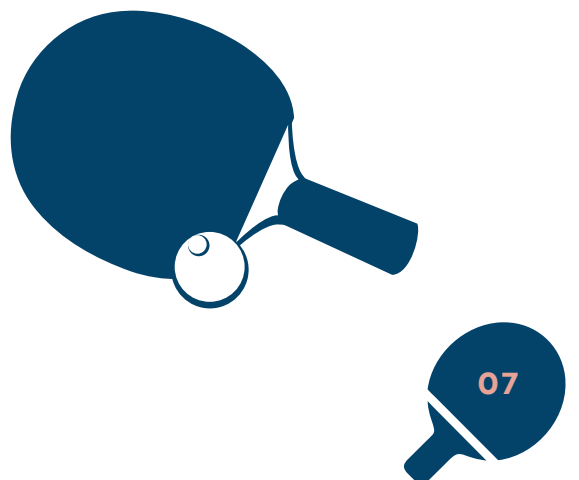
SCORE

06

MOVEMENT

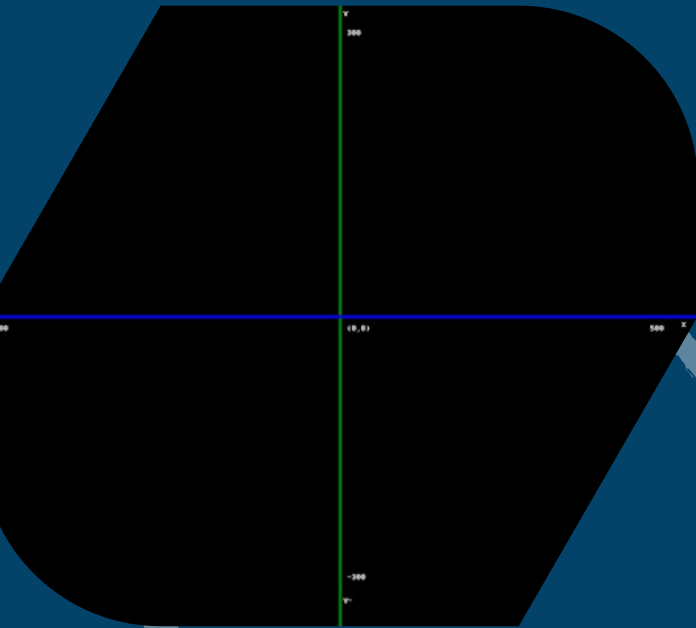
07

GAME LOOP



1

WINDOW



`turtle.Screen` => to create the screen for the library
`window.title("ping pong")` => to make the screen title
`window.setup(width=1000 , height=600)` => to make resolution of the screen into 600*800
`window.tracer(0)` => to turn off the delay of the screen
`window.bgcolor(0,0,.11)` => back ground color with RGB format
`window.listen()` => to interact with keyboard
`window.onkeypress` => with every keypress that assigned above turn on the movement function that we made

2 PLAYER 1&2

`Turtle . turtle ()` => Initializes a turtle object named "player_1" .
`Speed (0)` => Sets the speed to the fastest for instant rendering.

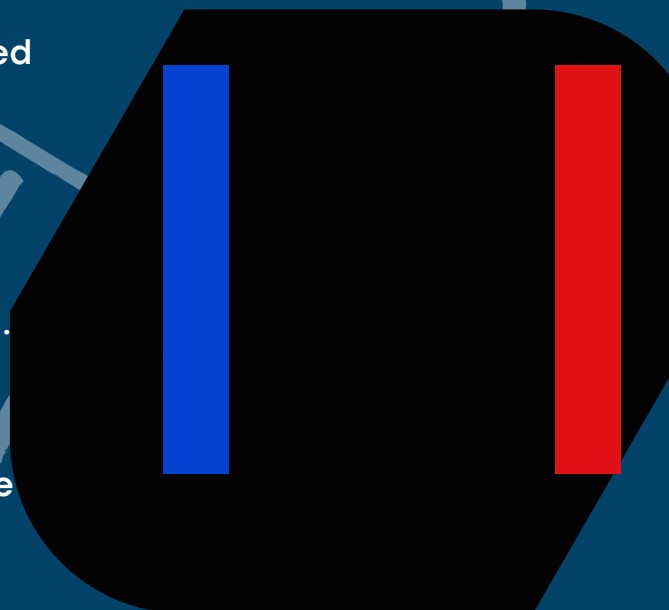
`Shape ("square")` => Defines the shape as a square

`Shapsize (stretch_len=1,stretch_wid=5)` =>stretches it to create a rectangular paddle.

`Color ("blue")` => Assigns a blue color to distinguish Player 1.

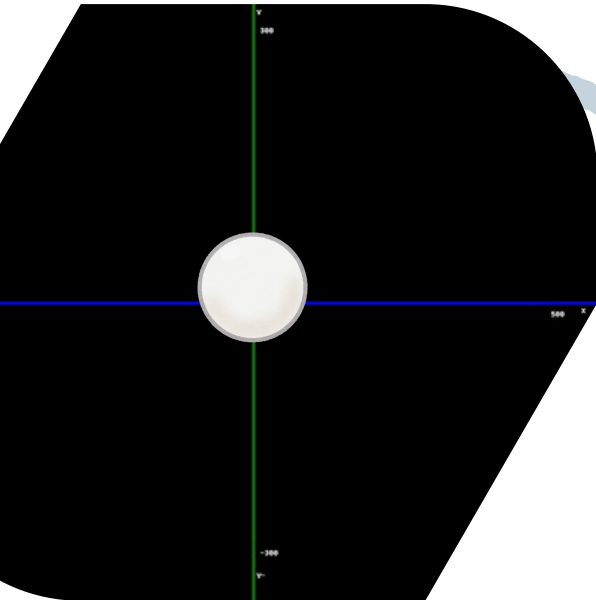
`Penup()` => Disables drawing while moving the turtle.

`Goto(x=-450,y=0)` => Positions the paddle on the left side of the screen at coordinates (-450, 0).



3

BALL



`shape("circle")` => Sets the shape of the ball to a circle.

`color("white")` => Sets the ball's color to white for clear visibility.

`shapesize(stretch_len=1, stretch_wid=1)`
=> Keeps the ball at the default size (20x20 pixels).

`goto(x=0, y=0)` => Places the ball at the center of the screen to start.

`penup()` => Prevents the ball from drawing lines as it moves.

`ball dx, ball_dy = 1, 1` => Sets the initial direction for horizontal and vertical movement.

`speed = 0.2` => Controls the overall speed of the ball.

4

CENTER LINE

`shape("square")` => Set the shape of the turtle to "square".

`color("white")` => Set the color of the turtle to white.

`shapesize(stretch_wid=25, stretch_len=0.1)` => Stretch the turtle to look like a tall, thin rectangle (like a center line).

`penup()` => Lift the pen so it doesn't draw when moving.

`goto(0, 0)` => Move the turtle to the center of the screen at coordinates (0, 0).



5 SCORE

`speed(0)` => Sets the turtle speed to the fastest (no animation delay)

`color("white")` => Sets the color of the text to white

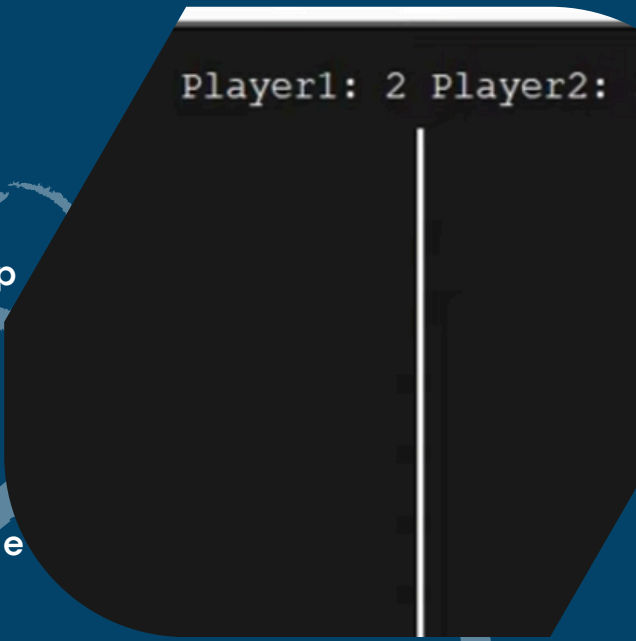
`penup()` => Lifts the pen to prevent drawing when the turtle moves

`goto(x=0, y=260)` => Moves the turtle to the top center of the screen

`write("player1: 0 player2: 0 ", align="center", font=("courier", 14, "normal"))` => Displays the initial score using a centered Courier font

`hideturtle()` => Hides the turtle icon so only the text is visible

`player_1_score, player_2_score = 0, 0`
=> Initializes both players' scores to 0



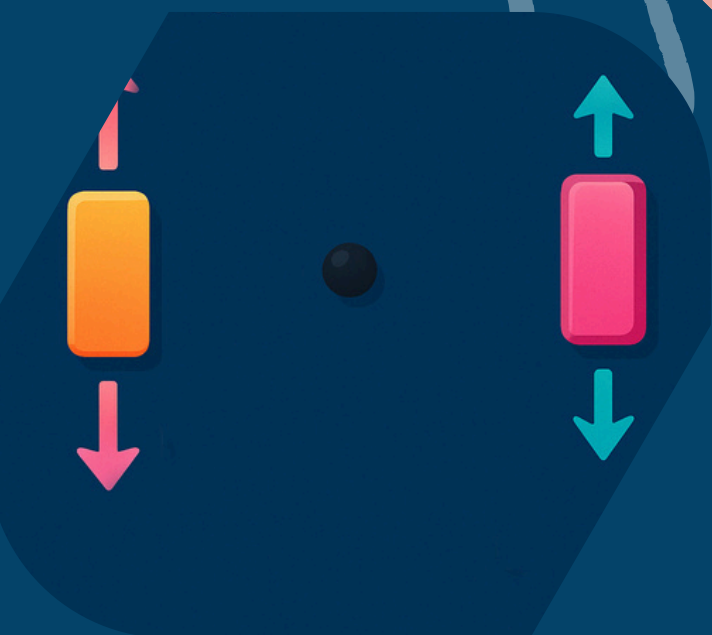
Player1: 2 Player2:

6 MOVEMENT

Player movement occurs along two axes (the x-axis and the y-axis).

For the player to move, the coordinates must be changed each time the user moves him, as follows: 1- The y-axis moves each player by 30 pixels per step, and in all cases, we do not need the player to move along the x-axis.

The movement code: "`sety()`" is used, where the player's position on the y-axis is changed by the specified amount without changing his position on the x-axis.



7 GAME LOOP

while True => Main game loop that keeps the game running continuously.

window.update() => Refreshes the game screen to show the latest positions.

ball.setx(...) / ball.sety(...) => Moves the ball based on its direction and speed.

ball.xcor() / ball.ycor() => gets the current x and y position of the ball

if ball.ycor() > 290 => Checks if the ball hits the top edge and makes it bounce back.

if ball.ycor() < -290 => Checks if the ball hits the bottom edge and makes it bounce back.

if ball.xcor() > 490 => If the ball goes out from the right, Player 1 scores.

if ball.xcor() < -490 => If the ball goes out from the left, Player 2 scores.

ball.goto(0, 0) => Resets the ball to the center after a score.

```

    ball.xcor() > -450 and (ball.ycor() > 290 or ball.ycor() < -290):
        ball_dx *= -1
        # Bounce on with player 2

    (ball.xcor() > 440 and ball.xcor() < 450 and (ball.ycor() > 290 or ball.ycor() < -290)) or ball.xcor() < -440:
        ball.setx(440)
        ball_dx *= -1

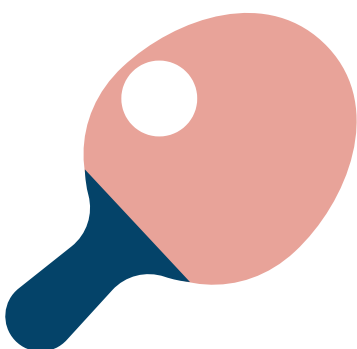
    # Score handling
    ball.xcor() > 490:
        ball.goto(x: 0, y: 0)
        ball_dx *= -1 # invert the direction of start
        score.clear()
        player_1_score += 1
        score.write(arg: f"player1: {player_1_score} player2: {player_2_score}")

    ball.xcor() < -490:
        ball.goto(x: 0, y: 0)
        ball_dx *= 1
        score.clear()
        player_2_score += 1
        score.write(arg: f"player1: {player_1_score} player2: {player_2_score}")

```

CHALLENGES AND SOLUTIONS

CHALLENGE	SOLUTION
Detecting precise collision between the ball and the paddles.	Implemented distance checking and boundary conditions to improve collision accuracy.
Smoothing paddle movement using key bindings.	Used Turtle's onkeypress() method for real-time paddle control.
Synchronizing the ball's speed and direction after collisions.	Adjusted ball velocity dynamically based on collision points.



CONCLUSION

The Ping Pong game project provided valuable hands-on experience in event-driven programming and GUI development using Python's Turtle module. Throughout the development process, we improved our problem-solving abilities and gained a deeper understanding of real-time object interactions.

For future enhancements, the game could be expanded by incorporating sound effects, multiple difficulty levels, enhanced graphics, and an AI-controlled opponent to allow single-player gameplay. These additions would further enrich the user experience and showcase more advanced programming techniques.

